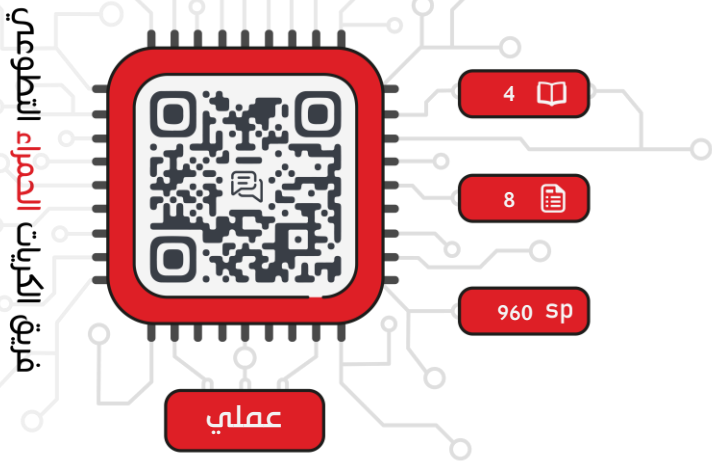


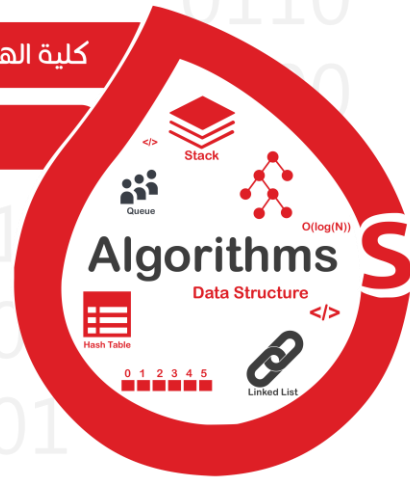


كلية الهندسة المعلوماتية

السنة الثانية

Stack & Queue

عملي مشترك



RB Informatics; 2/12/2025

خوارزميات 1

سنكمل رحلتنا في ال Data structure التي كنا قد بدأناها في المحاضرة السابقة مع: LinkedList.

Stack

وتسمّى أيضاً المُكدّس أو الأطباق.

يعمل بنظام ال ((LIFO)) "Last in, First out" أي العناصر التي تمت إضافتها أخيراً هي الأولى التي ستتم إزالتها. حيث سنركّز دوماً على القمة top وهو آخر عنصر تمت إضافته أو تكون قيمته 1- في حال كان stack فارغ.

هناك عمليات أساسية:

1. push() ويكون بهذا الاسم حصراً في أغلب المراجع لأنه عند إضافة عنصر يحصل دفع للعناصر في المكدّس، وإن كان المكدّس ممتلئ يحدث فيض أو تجاوز سعة stack overflow
2. pop() لإخراج العناصر بعكس الترتيب الذي دخلت فيه، فإذا كان المكدّس فارغاً يحدث ما يسمّى بالـ stack underflow
3. isEmpty() لمعرفة حالة ال stack إذا كان فارغ تعيد قيمة true وإلا تعيد قيمة false .
4. peek() للحصول على قيمة العنصر العلوي من دون إزالته.

تعقيد هذه العمليات $O(1)$ بسبب استخدام عمليات ثابتة -لا يوجد حلقات تكرارية- كما سنرى.

إضاءة آرسيزية:

- العودية Recursion: تحدث عن طريق ال stack، حيث في كل مرة تستدعي فيها دالة نفسها (أو دالة أخرى) يتم تخزين حالة الاستدعاء الحالية (مثل المتغيرات، موقع التنفيذ) على مكس النداء (Call Stack) في الذاكرة، بحيث عند انتهاء الدالة يتم الرجوع للحالة السابقة من المكس وتنفيذها. فالمكدس هو المسؤول عن إدارة تسلسل الاستدعاءات والعودة منها بطريقة منظمة (يأخذ قمة المكس).
- لذلك عند الاستدعاءات اللانهائية يحدث ما يسمى stack overflow، مما يؤدي إلى نفاذ الذاكرة المخصصة للتابع.

نستطيع بناء stack بطريقتين:

1. عن طريق Array:

حيث سيمثل top: index آخر عنصر تمت إضافته لذلك تكون قيمته الابتدائية 1- عندما تكون stack فارغة لأن القيمة 0 تمثل أول عنصر في المصفوفة عند الإضافة.

في الكود أدناه عرفنا static final int max؛ الحد الأقصى الذي يمكن أن تستوعبه المصفوفة ولا يمكن التعديل عليه (final)، و من الممكن أن نحدده عند إنشاء stack بإعطاء القيمة المناسبة .

```
public class Stack {
    static final int max = 1000; //Maximum value that can be added
    to the stack
    int[] a = new int[max];
    int top;

    public Stack() {
        top=-1;
    }

    boolean isEmpty() {
        return (top < 0);
    }

    void push(int x) {
        if(top>=max-1) System.out.println("StackOverFlow");
        else{
            a[++top]=x; //a[top]=x; top++;
            System.out.println(x+" pushed into stack");
        }
    }

    int pop() {
        if(top<0) {
            System.out.println("Stack under flow");
            return 0;
        }
    }
}
```

```

    } else {
        int x = a[--top]; // x=a[top]; top--;
        return x;
    }
}
int peek() {
    if (top < 0) {
        System.out.println("Stack under flow");
        return 0;
    } else return a[top];
}
}

```

محاسن هذه الطريقة: سهولة التطبيق ولا تستهلك الكثير في الذاكرة لعدم استخدام المؤشرات.
المساوي: غير ديناميكية، نحجز مساحة محددة للمصفوفة غير قابلة للتوسعة أو التقليل.

2. عن طريق LinkedList:

top تمثل مؤشر على رأس linkedList.

```

class Stack {

    Node top;

    public Stack() {
        top = null;
    }

    static class Node {

        int data;
        Node next;

        public Node(int data) {
            this.data = data;
        }
    }

    boolean isEmpty() {
        if (top == null) {
            return true;
        }
        return false;
    }
}

```

```
void push(int data) {
    Node n = new Node(data);
    if (top == null) {
        top = n;
        return;
    }
    n.next = top;
    top = n;
}

public int pop() {
    int popped = Integer.MIN_VALUE;
    if (top == null) {
        System.out.println("stack under flow");
    } else {
        popped = top.data;
        top = top.next;
    }
    return popped;
}

int peek() {
    if (top == null) {
        System.out.println("stack under flow");
        return Integer.MIN_VALUE;
    }
    else
        return top.data;
}
}
```

- ملاحظة 1: Integer.MIN_VALUE في جافا هو ثابت يمثل أقل قيمة ممكنة يمكن تخزينها في متغير من نوع int يستخدم هذا الثابت لتحديد الحد الأدنى للقيم الصحيحة، وغالبًا ما يُستخدم في البرمجة للمقارنات كما في مثالنا.
- ملاحظة 2: لا يوجد overflow عند تطبيق linkedList لأن حجمها ديناميكي غير ثابت، وهذا من مميزات هذه الطريقة.

أما مساوئ هذه الطريقة: تحتاج إلى استخدام جزء أكبر من الذاكرة نظرًا لاستخدام المؤشرات.

الأرتال Queues

تشبه المكادس "Stacks" في كونها بنى معطيات خطية تتبع نمطاً معيناً في ترتيب العناصر وإجراء العمليات عليها.

- يعمل بنظام ال ((FIFO)) "First in, First out" أي: الأولوية للدخول أولاً، العنصر الذي دخل أولاً يخرج أولاً.
- تتمثل في سياق الصف في الحياة الواقعية؛ حيث يتم تشكيل طابور من الناس و يتم حذفهم وفقاً لترتيب وصولهم.
- الفرق بين الرتل والمكدس هو في خروج العناصر، إذ أن آخر عنصر يُضاف إلى المكدس هو الذي سيخرج أولاً، أما آخر عنصر يُضاف إلى الرتل فهو الذي سيخرج آخرًا.

توابع:

1. الإضافة Enqueue: تضيف هذه العملية عنصراً إلى الرتل، وإن كان الرتل ممتلئاً فتسمى الحالة حينئذٍ بحالة الفيض (تجاوز السعة Over Flow).
2. الازالة Dequeue: تزيل هذه العملية عنصراً من الرتل، وتزال العناصر من الرتل بنفس ترتيب إضافتها، وإن كان الرتل فارغاً فتسمى الحالة حينئذٍ بحالة underflow كما سبق في ال stack.
3. FrontQ: يمكن من خلال هذا العملية الحصول على العنصر الأمامي في الرتل.
4. RearQ: يمكن من خلال هذه العملية الحصول على العنصر الخلفي في الرتل.

التعقيد الزمني لجميع العمليات الخاصة بالأرتال هو $O(1)$ ، وذلك لعدم استخدام الحلقات التكرارية لدى تنفيذ هذه العمليات.

إنشاء الأرتال باستخدام مصفوفات:

يجب متابعة فهرسين من فهارس المصفوفة عند استخدامها في إنشاء الأرتال وهما الفهرسان الأول والآخر، إذ تضاف العناصر إلى نهاية المصفوفة وتزال من بدايتها. قد يبدو الأمر بسيطاً، ولكن زيادة الفهرسين الأول والآخر قد يتسبب في حدوث مشكلة هي وصول المؤشر الأول إلى نهاية المصفوفة؛ ولحل هذه المشكلة يجب زيادة الفهرسين الأول والآخر بطريقة دائرية.



```
public class Queue {
    int front=0, rear, size=0, capacity;
    int []a;

    public Queue(int capacity) {
        this.capacity = capacity;
        rear=capacity-1;
        a=new int[this.capacity];
    }
    boolean isFull(Queue q) {
        return (q.size==q.capacity);
    }
    boolean isEmpty(Queue q) {
        return (q.size==0);
    }
    void enqueue(int item){
        if(isFull(this))return;
        this.rear=(this.rear+1)%this.capacity;
        this.a[this.rear]=item;
        this.size++;
        System.out.println(item+" is enqueued ");
    }
    int dequeue() {
        if(isEmpty(this))return Integer.MIN_VALUE;
        int item=this.a[this.front];
        this.front=(this.front+1)%this.capacity;
        this.size--;
        return item;
    }
    int front() {
        if(isEmpty(this))return Integer.MIN_VALUE;
        return this.a[this.front];
    }
    int rear() {
        if(isFull(this))return Integer.MIN_VALUE;
        return this.a[this.rear];
    }
}
```

- الـ capacity هي حجم المصفوفة يتم إعطاؤها عند إنشاء Queue جديدة أما size؛ فهي لمعرفة عدد العناصر الحالية في المصفوفة ومقارنتها مع capacity.
- والآن مع تابع enqueue : "بداية" - و خلوها بيناتنا - شيل الـ this من خوارزمتك بس لحتى تشير للكائن مشان تفهم الفكرة... و بتبقى بترجعها على الكود بس هلا الأهم نرجع للفصحى".
- $rear = (rear + 1) \% capacity$
- هذه التعليمة ليست معقدة كما تعتقد، ولكنها خطوة مهمة فهي أساس عمل الرتل الدائري سبق أن ذكرنا: أن المصفوفة بطريقة الأرتال يتم تعبئتها بطريقة دائرية أي عندما نصل لنهاية المصفوفة نعود للبداية مجدداً في حال خروج عناصر من الرتل .
- في أول مرة سيكون $rear = capacity - 1$ ثم نضيف 1 فيصبح مساوياً لـ capacity ويكون باقي قسمته عليها 0 أي نصل لأول index، ثم في المرة التالية نصل لـ index الثاني وهكذا حتى يمتلئ الرتل وفي حال أزلنا عنصر أو أكثر (لا يكون الإخراج في طريقة الأرتال إلا من البداية - الأقدم دخولاً-) عندئذٍ نتمكن من الإضافة في البداية عند الـ index صفر.
- هكذا نستغل كل مساحة المصفوفة دون إضاعة مكان فارغ في حال front تحرك.
- وكذلك عند حذف عنصر نزيد المؤشر front بطريقة دائرية أيضاً.

إنشاء الأرتال باستخدام linkedList:

```
public class Queue {
    Node front, rear;
    public class Node {
        int data;
        Node next;

        public Node(int data) {
            this.data = data;
            next = null;
        }
    }

    //constructor
    public Queue() {
        this.front = this.rear = null;
    }
}
```

```
void enqueue(int item){
    Node n=new Node(item);
    if(this.rear==null){
        this.front=this.rear=n;
        return;
    }
    this.rear.next=n; //الإضافة تكون من النهاية في الأرتال
    this.rear=n;
}

Node dequeue() {
    if(this.front==null) return null;
    Node n=this.front;
    this.front=this.front.next;
    if(this.front==null) this.rear=null; // في هذه الحالة أصبح
    الرتل فارغ
    return n;
}
}
```

One day... all of your dreams will come true.

